

Principles of Statistical Data Mining

Autumn 2025

Graded Homework Assignment 3

“We are what we repeatedly do. Excellence therefore
is not an act but a habit”

– Aristotle

Professor: Dr. Ernest Fokoué

Due Date: November 16, 2025 · 11:59 PM

Note: This assignment explores the beautiful geometry of SVMs and the elegant recursive partitioning of trees.
Embrace both the mathematical foundations and practical implementations.

Elements of the Assignment

Dear Data Mining Explorers,

This assignment takes you deep into two powerful paradigms: the geometric elegance of Support Vector Machines and the intuitive recursive partitioning of tree-based methods. You'll explore both regression and classification tasks, balancing theoretical understanding with practical implementation.

Specific Instructions:

- The total number of points for this assignment is **150** points.
- You **MUST** submit a single, beautifully typeset PDF report ideally generated by knitting your RMarkdown file.
- You **MUST** show all relevant work and reasoning.
- All computer code should be provided through the RMarkdown file.
- **The video component is mandatory!**
- Creativity and deep insight are expected, and will be rewarded!
- **Mindless copying and pasting of LLM outputs for any question will incur a huge ZERO!**
- This assignment may appear long, but it is NOT. It is deliberately designed to help you review and deepen your understanding of the granular details of these important concepts. It is an application of the wisdom of Aristotle.
- You are being given 14 full days to complete this assignment, and that should suffice.

Now, let the learning begin!

Exercise 1: Theoretical Foundations (50 points)

Challenge 1. *True mastery requires understanding both the mathematical foundations and the geometric intuitions behind learning algorithms. Your task: derive, prove, and visualize the core concepts.*

Consider the classical statistical supervised learning task with a given data set

$$\mathcal{D}_n = \{(\mathbf{x}_i, y_i) \sim p_{\mathbf{xy}}(x, y), \mathbf{x}_i \in \mathcal{X}, y_i \in \mathcal{Y}, i = 1, \dots, n\},$$

from some unknown joint probability density function $p_{\mathbf{xy}}(x, y)$.

Part A: Support Vector Machines (30 points)

Consider the soft-margin SVM optimization problem for binary classification with class labels in $\{-1, +1\}$ formulated as follows:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

subject to:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad i = 1, \dots, n$$

The Primal-Dual Gap and a "Broken" SVM (10 points)

A student, Alice, is implementing a Soft-Margin SVM solver. She derives the Lagrangian and the KKT conditions correctly. However, due to a bug, her solver fails to enforce the constraint $\alpha_i \leq C$ for the Lagrange multipliers in the dual problem. Her "broken" dual problem is:

$$\hat{\alpha} = \underset{\alpha \in \mathbb{R}_+^n}{\operatorname{argmax}} \left\{ \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j \right\}$$

subject to $\sum_{i=1}^n \alpha_i y_i = 0$ and $\alpha_i \geq 0$ for $i = 1, \dots, n$.

- (a) **(2 points)** Clearly specify and write down what is missing in Alice's dual formulation.
- (b) **(3 points)** Explain the geometric interpretation of the primal SVM objective: to find a hyperplane that maximizes the margin while allowing for slack variables ξ_i . How does the parameter C control the trade-off in this geometric view?
- (c) **(3 points)** Now, consider Alice's broken dual. Assume the data is linearly separable. What will happen to the optimal α_i for a support vector in this broken formulation as the optimization runs? What does this imply for the resulting weight vector $\mathbf{w} = \sum \alpha_i y_i \mathbf{x}_i$?
- (d) **(2 points)** Connect your answer in (b) to the KKT condition involving the slack variable ξ_i and the multiplier μ_i (where $\mu_i = C - \alpha_i$ in the correct formulation). Explain why the missing constraint $\alpha_i \leq C$ breaks the model's ability to handle outliers and misclassified points.

The Kernel Trick: From Theory to Practice (10 points)

- (a)
- (3 points)**
- Prove that the prediction function for a kernelized SVM,

$$\hat{f}(\mathbf{x}_{\text{new}}) = \text{sign} \left(\sum_{i \in SV} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}_{\text{new}}) + b \right),$$

can be computed *without ever explicitly calculating* $\phi(\mathbf{x})$ for any point, where ϕ is the feature map corresponding to the kernel K .

- (b)
- (4 points)**
- Consider a custom kernel function defined as

$$K(\mathbf{u}, \mathbf{v}) = (1 + \mathbf{u}^\top \mathbf{v})^3$$

- (i) What is the dimensionality of the implicit feature space $\phi(\mathbf{u})$ for this kernel? Justify your answer.
- (ii) Write out the explicit formula for *one specific component* of the feature vector $\phi(\mathbf{u})$ for a 2-dimensional input $\mathbf{u} = (u_1, u_2)$. This demonstrates you understand the relationship between the kernel and the feature map.
- (c) **(3 points)** A common claim is that "the RBF kernel can map data to an infinite-dimensional feature space." Explain intuitively why this is both a great strength and a significant danger, linking your answer to the concept of model complexity and the bias-variance trade-off.

Support Vector Regression: An Asymmetric Twist (10 points)

We now turn to Support Vector Regression Learning corresponding to the use of the large margin paradigm when the outspace $\mathcal{Y} \subseteq \mathbb{R}$. The standard ϵ -insensitive loss in Support Vector Regression (SVR) is symmetric: it penalizes absolute deviations greater than ϵ equally, regardless of sign.

$$L_\epsilon(y, f(\mathbf{x})) = \max(0, |y - f(\mathbf{x})| - \epsilon)$$

Imagine a financial forecasting task where overestimating a company's value ($f(\mathbf{x}) > y + \epsilon$) is much more costly than underestimating it ($f(\mathbf{x}) < y - \epsilon$).

- (a) **(4 points)** Propose a mathematically sound *asymmetric loss function* for SVR to handle this scenario. Sketch its plot.
- (b) **(3 points)** Formulate the *primal* optimization problem for an SVR using your proposed asymmetric loss. You do not need to derive the dual, but clearly define all variables and constraints.
- (c) **(3 points)** Propose and write down the mathematical expression of $\hat{f}_{\text{svr}}(\mathbf{x}_{\text{new}})$ in terms of the kernel and the Lagrange multipliers.

Part B: Connecting Learning Paradigms (20 points)

The Grand Unified View: Loss + Penalty (10 points)

Many statistical learning algorithms can be expressed in the form:

$$\min_{f \in \mathcal{H}} \sum_{i=1}^n L(y_i, f(\mathbf{x}_i)) + \lambda J(f)$$

where L is a loss function and $J(f)$ is a regularizer.

- (a) **(6 points)** Complete the following table, specifying the Loss $L(\cdot, \cdot)$ and Regularizer $J(f)$ for each method.

Method	Loss $L(y, f(\mathbf{x}))$	Regularizer $J(f)$	Function Class $\mathcal{H}$
Ridge Regression	$(y - f(\mathbf{x}))^2$	$\ \beta\ _2^2$	Linear functions
Lasso			
SVM (Primal)			
Logistic Regression			

- (b) **(4 points)** Based on your table, explain the fundamental philosophical difference between the "loss" used in SVM classification and the "loss" used in Logistic Regression. How does this difference manifest itself in the resulting models (e.g., in their sensitivity to outliers)?

The Dimensionality Curse and Blessing (10 points)

Consider the RBF kernel $K(\mathbf{u}, \mathbf{v}) = \exp(-\gamma \|\mathbf{u} - \mathbf{v}\|^2)$.

- (a) **(3 points)** In high-dimensional spaces ($p \gg n$), explain the phenomenon known as "distance concentration," where many pairs of points become almost equidistant. What is the implication of this for the RBF kernel matrix as p grows very large?
- (b) **(4 points)** Propose a theoretical argument for why, in such high-dimensional settings, an RBF-SVM might struggle to learn a meaningful pattern without very careful tuning of γ . (*Hint:* Think about the kernel matrix becoming increasingly similar to the identity matrix).
- (c) **(3 points)** Surprisingly, SVMs often work well in practice even when $p > n$. Connect this to the "blessing of dimensionality" argument: that in high dimensions, data is often inherently linearly separable. Explain briefly why this might be the case.

Fundamental Insight 1. *The beauty of SVMs lies in their geometric interpretation and convex optimization, while trees excel in interpretability and handling complex interactions. Understanding both gives you a complete toolkit.*

Exercise 2: Applied Regression Learning (50 points)

Challenge 2. *Now apply your theoretical knowledge to real data. Compare tree-based methods with SVR on a challenging regression task, evaluating estimation accuracy, prediction performance, and model interpretability.*

The dataset for statistical learning task is `datafls` from `library(BMS)`, and the Response variable: `y` (continuous outcome).

Part A: Data Exploration and Preparation (15 points)

1. (5 points) Data Understanding

- (a) Load the data and provide summary statistics.
- (b) Create visualizations showing relationships between predictors and response.
- (c) Check for missing values and outliers.

2. (5 points) Feature Engineering

- (a) Create any relevant interaction terms or polynomial features.
- (b) Standardize features for SVR (explain why this is crucial).
- (c) Split data into training (70%), validation (15%), and test (15%) sets.

3. (5 points) Baseline Models

- (a) Fit a linear regression baseline.
- (b) Fit a single decision tree (no pruning).
- (c) Report baseline performance on validation set.

Part B: Support Vector Regression (15 points)

1. (5 points) Model Training

- (a) Train SVR models with linear, polynomial, and RBF kernels.
- (b) Use cross-validation to tune hyperparameters (C , ϵ , kernel parameters).
- (c) Plot validation performance vs. hyperparameter values.

2. (5 points) Model Evaluation

- (a) Compare test performance of different kernels.
- (b) Analyze residuals and identify patterns.
- (c) Discuss computational requirements of different kernels.

3. (5 points) Interpretation

- (a) How many support vectors were selected?
- (b) What does this tell you about the complexity of the solution?
- (c) Discuss challenges in interpreting SVR models.

Part C: Tree-Based Regression (10 points)**1. (4 points) Single Trees**

- (a) Train a pruned decision tree using cross-validation.
- (b) Visualize the tree structure and important splits.
- (c) Compute feature importance scores.

2. (3 points) Ensemble Methods

- (a) Train a random forest and gradient boosting machine.
- (b) Tune hyperparameters using validation set performance.
- (c) Compare out-of-bag error with validation error.

3. (3 points) Interpretation

- (a) Create partial dependence plots for important features.
- (b) Compare feature importance across different tree methods.
- (c) Discuss the trade-off between interpretability and performance.

Part D: Comprehensive Comparison (10 points)**1. (3 points) Performance Metrics**

- (a) Create a table comparing all methods on test set RMSE, MAE, and R^2 .
- (b) Perform statistical significance testing of performance differences.
- (c) Analyze which method works best for different regions of feature space.

2. (3 points) Practical Considerations

- (a) Compare training time and prediction speed.
- (b) Discuss robustness to outliers and noise.
- (c) Analyze sensitivity to hyperparameter choices.

3. (4 points) Recommendations

- (a) Based on your analysis, which method would you recommend for this dataset?
- (b) Justify your recommendation considering accuracy, interpretability, and computational cost.
- (c) What would be your next steps for improvement?

Exercise 3: Applied Classification Learning (40 points)

Challenge 3. *The MNIST dataset provides a perfect playground for comparing classification paradigms. Your task: build, evaluate, and interpret both SVM and tree-based classifiers on this iconic dataset.*

Dataset: MNIST Handwritten Digits

10-class classification problem: digits 0-9

Part A: Data Preparation and Exploration (10 points)

1. (3 points) Data Loading and Inspection

- (a) Load MNIST data (training: 60,000 samples, test: 10,000 samples).
- (b) Visualize sample images from each class.
- (c) Analyze class distribution and pixel value statistics.

2. (3 points) Feature Engineering

- (a) Implement dimensionality reduction (PCA) and visualize in 2D.
- (b) Create derived features (e.g., pixel intensity statistics, geometric moments).
- (c) Compare raw pixels vs. engineered features in preliminary models.

3. (4 points) Data Subsetting

- (a) Create balanced subsets for efficient experimentation.
- (b) For computational efficiency, work with 10,000 training samples initially.
- (c) Create validation split for hyperparameter tuning.

Part B: Support Vector Classification (10 points)

1. (3 points) Binary Classification

- (a) Implement one-vs-rest strategy for multi-class classification.
- (b) Train SVM with RBF kernel for digit "3" vs. all others.
- (c) Analyze decision boundary and support vectors.

2. (4 points) Multi-class Classification

- (a) Train multi-class SVM with RBF kernel.
- (b) Use cross-validation to tune C and γ .
- (c) Analyze confusion matrix and per-class performance.
- (d) Visualize misclassified examples and identify patterns.

3. (3 points) Advanced Kernels

- (a) Experiment with polynomial kernels.
- (b) Discuss which kernel works best for image data and why.
- (c) Analyze computational complexity vs. performance.

Part C: Tree-Based Classification (10 points)**1. (3 points) Single Decision Tree**

- (a) Train a decision tree with various depth limits.
- (b) Visualize the tree structure for a binary subproblem.
- (c) Analyze which pixels are most important for classification.

2. (4 points) Ensemble Methods

- (a) Train random forest with various numbers of trees.
- (b) Implement gradient boosting with early stopping.
- (c) Compare out-of-bag error with cross-validation error.
- (d) Analyze feature importance across ensemble methods.

3. (3 points) Interpretation

- (a) Create partial dependence plots for important pixels.
- (b) Visualize what the tree "sees" when classifying different digits.
- (c) Compare interpretability with SVM models.

Part D: Advanced Analysis and Comparison (10 points)**1. (3 points) Performance Deep Dive**

- (a) Compare all methods on full test set (10,000 samples).
- (b) Create precision-recall curves for each class.
- (c) Analyze which digits are most challenging for each method.

2. (3 points) Computational Analysis

- (a) Compare training time and memory usage.
- (b) Analyze prediction speed on test set.
- (c) Discuss scalability to full dataset (60,000 training samples).

3. (4 points) Hybrid Approaches

- (a) Experiment with SVM on PCA-reduced features.
- (b) Try tree-based methods on raw pixels vs. engineered features.
- (c) Propose and test one innovative hybrid approach.

Exercise 4: The Master's Reflection (10 points)

Challenge 4. *True data mining mastery requires not just technical skill, but the ability to reflect on your learning journey, articulate insights, and communicate complex ideas effectively.*

Your Oral Defense

Create a 7-minute video entitled “**From Theory to Practice: My Journey with Learning Machines**”.

Your Defense Structure

1. (2 minutes) The Philosophical Foundation

- What distinguishes tree-based methods from kernel methods philosophically?
- How has your understanding of the bias-variance tradeoff evolved?
- What is the most elegant mathematical insight you gained?

2. (3 minutes) Practical Wisdom

- Share your most surprising finding from the regression task.
- Discuss key insights from the MNIST classification challenge.
- What practical advice would you give someone starting with these methods?

3. (2 minutes) The Big Picture

- When would you prefer SVMs over tree-based methods in practice?
- What are the current limitations of both approaches?
- How has this assignment changed your approach to model selection?

Technical Requirements

- Face visible (picture-in-picture is excellent)
- Your completed PDF report displayed as you comment
- Upload to YouTube with access for course staff
- Include link in your submission

Fundamental Insight 2. *The mark of true expertise is not just knowing how to apply methods, but understanding when and why to choose them, and being able to communicate these insights clearly to others.*

Appendix: R/Python Code Template

```
# Load required packages
library(e1071)      # SVM implementation
library(rpart)      # Decision trees
library(randomForest) # Random forests
library(xgboost)     # Gradient boosting
library(caret)       # Machine learning utilities
library(pROC)        # ROC curves
library(ggplot2)     # Visualization

# =====
# EXERCISE 2: Applied Regression Learning
# =====

# Load data
library(BMS)
data(datafls)

# Data preparation
X <- datafls[, -which(names(datafls) == "y")]
y <- datafls$y

# Train-test split
set.seed(747)
train_idx <- sample(1:nrow(datafls), 0.7 * nrow(datafls))
X_train <- X[train_idx, ]
y_train <- y[train_idx]
X_test <- X[-train_idx, ]
y_test <- y[-train_idx]

# Support Vector Regression
svr_model <- svm(X_train, y_train, kernel = "radial",
                cost = 10, epsilon = 0.1)
svr_pred <- predict(svr_model, X_test)
svr_rmse <- sqrt(mean((svr_pred - y_test)^2))

# Random Forest
rf_model <- randomForest(X_train, y_train, ntree = 500)
rf_pred <- predict(rf_model, X_test)
rf_rmse <- sqrt(mean((rf_pred - y_test)^2))

# =====
# EXERCISE 3: MNIST Classification
```

```
# =====  
  
# Load MNIST data (example using keras)  
library(keras)  
mnist <- dataset_mnist()  
X_train <- mnist$train$x  
y_train <- mnist$train$y  
X_test <- mnist$test$x  
y_test <- mnist$test$y  
  
# Preprocess: flatten images and normalize  
X_train <- array_reshape(X_train, c(nrow(X_train), 784)) / 255  
X_test <- array_reshape(X_test, c(nrow(X_test), 784)) / 255  
  
# SVM Classification (subset for efficiency)  
svm_model <- svm(X_train[1:10000, ], as.factor(y_train[1:10000]),  
                 kernel = "radial", cost = 10)  
svm_pred <- predict(svm_model, X_test)  
svm_accuracy <- mean(svm_pred == y_test)  
  
# Random Forest (subset for efficiency)  
rf_model <- randomForest(X_train[1:10000, ], as.factor(y_train[1:10000]),  
                         ntree = 100)  
rf_pred <- predict(rf_model, X_test)  
rf_accuracy <- mean(rf_pred == y_test)
```